

# ToolFactory: Automating Tool Generation by Leveraging LLM to Understand REST API Documentations

Xinyi Ni<sup>1</sup>, Qiuyang Wang<sup>1</sup>, Yukun Zhang<sup>1</sup>, Pengyu Hong<sup>1</sup>

<sup>1</sup>Computer Science, Michtom School of Computer Science, Brandeis University  
{xinyini, qiuyangwang, yukunzhang, hongpeng}@brandeis.edu

## Abstract

LLM-based tool agents offer natural language interfaces, enabling users to seamlessly interact with computing services. While REST APIs are valuable resources for building such agents, they must first be transformed into AI-compatible tools. Automatically generating AI-compatible tools from REST API documents can greatly streamline tool agent development and minimize user learning curves. However, API documentation often suffers from a lack of standardization, inconsistent schemas, and incomplete information. To address these issues, we developed **ToolFactory**, an open-source pipeline for automating tool generation from unstructured API documents. To enhance the reliability of the developed tools, we implemented an evaluation method to diagnose errors. Furthermore, we built a knowledge base of verified tools, which we leveraged to infer missing information from poorly documented APIs. We developed the API Extraction Benchmark, comprising 167 API documents and 744 endpoints in various formats, and designed a JSON schema to annotate them. This annotated dataset was utilized to train and validate ToolFactory. The experimental results highlight the effectiveness of ToolFactory. We also demonstrated ToolFactory by creating a domain-specific AI agent for glycomaterials research. ToolFactory exhibits significant potential for facilitating the seamless integration of scientific REST APIs into AI workflows.

## 1 Introduction

Agents based on large language models (LLM) [1] are rapidly proving their versatility and universal applicability in various fields, such as scientific research [2], healthcare [3], finance [4], and so

on. These agents show impressive reasoning and problem-solving capabilities, addressing real-world challenges. Tool agents, built on LLMs, aim at interacting autonomously with existing software or web services. For example, ChemCrow [5], a tool agent recently developed for chemistry research, leverages domain-specific software to complete various tasks, such as searching, instruction, analysis, and prediction. However, constructing such agents often requires significant software engineering effort and domain knowledge. On the other hand, a tool agent can utilize existing **REST APIs**, especially those associated well-designed schemas that clearly explain in machine-understandable ways how the APIs should be used correctly and precisely. Developing such tool agents involves minimal programming effort. Mastering REST APIs enables tool agents to access a variety of data and computing services. For example, Tool-Llama [6] utilizes over 16,000 APIs from RapidAPI, creating a comprehensive toolkit to address various problems in multiple application domains. However, it relies on API platforms for generating API tools, which cannot be transferred to build research AI agent.

Several obstacles hinder the development of tool agents in research domains. Unlike commercial APIs, scientific APIs often lack comprehensive documentation and rarely adhere to standardized schemas. Even when a schema is provided, it may be incomplete or missing critical information, complicating the creation of a generalized tool generation process. Additionally, scientific APIs and their documentation are typically updated less frequently than commercial counterparts, necessitating extensive validation and refinement before they can be effectively utilized.

We consider it essential to develop an automated pipeline for generating agent-compatible tools in research domains. For scientific developers, such a pipeline enables the rapid creation of AI-usable tools, provided that APIs and their documentations are available in natural language. This improves development efficiency and allows developers to focus on

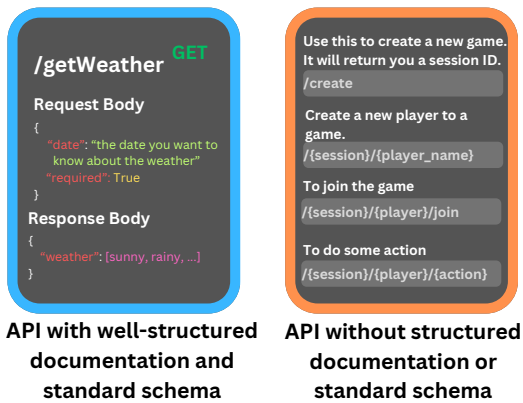


Figure 1: **The API Extraction Benchmark includes API documents with varying levels of structures.** For example, the API document example shown in the left follows a standardized schema. The one in the right is less structured as several fields are described in free-text. Our dataset prioritizes API variety and emphasizes the less structured cases. The diverse document structures in our dataset necessitate a general tool generation pipeline capable of processing various document formats effectively.

their core research tasks. For end users, AI agents offer a natural language interface to interact with APIs, significantly reducing the learning curve and making tools more accessible. For the AI agent community, an automated pipeline allows for seamless integration of any available APIs, thereby greatly enhancing the versatility and capability of AI agents.

To address the limitations in tool agent development, we propose an open-source pipeline, **ToolFactory**<sup>1</sup> (Figure 2), which autonomously generates tools from any REST API documentation written in natural language, eliminating the need for human intervention. To demonstrate our approach, we also introduced an **API Extraction Benchmark** specifically constructed for automatic tool generation:

- **Data collection:** We gathered 167 free API documentations from APIList.com. **Unlike previous benchmarks, we focused on APIs with diverse documentation formats** (see Figure 1 for examples). This deliberate choice was made to ensure generalizability of our approach, as relying too heavily on APIs with uniform schemas (e.g., OpenAPI spec, RapidAPI schema) could limit the applicability of our approach. Documentation quality varies significantly among open-source APIs, which introduces challenges in understanding the APIs. We selected APIs that do not require authentication to simplify the evaluation framework and to avoid potential safety concerns. By using authentication-free APIs, we isolate the key

challenge of extracting information from diverse documentation styles. In practical applications, authentication keys can be applied manually and incorporated into the AI-usable tools as part of their configuration.

- **Schema design:** Since the collected API documents are of various structures, we established a minimum required JSON schema (Appendix A) for standardizing the structured information that should be extracted from the documents, facilitating autonomous tool generation. Unlike common REST API schemas such as OpenAPI, our schema excludes the definition of API responses, as these are not always provided in documentation and do not directly impact API usage. In addition, we employ an LLM-based evaluator to handle different responses and classify possible errors (Section 4.3), as a complement to our schema design.
- **API document annotation:** We utilized GPT-4o in structured mode to produce standardized and well-organized API information for the API documents in our dataset. The quality of the data was examined and verified by human experts.

Our contributions can be summarized as follows:

- Using the API extraction benchmark, we derived an open-source model, **APILLama**, by fine-tuning Llama 3 [7] for the API extraction task using prompt tuning [8]. Specifically, the model is trained to transform an API documentation into a JSON file that meets the predefined schema for the AI agent to perform as a tool, an example is provided in Appendix B.2. We used prompt tuning with a soft prompt in the embedding layer to guide extraction. The soft prompt efficiently encodes the JSON schema and instruction (about 572 tokens) into shorter length (20 trainable virtual tokens). This approach minimizes trainable parameters, reduces overfitting, and preserves performance. Our experiment (Section 5.1) shows that APILLama achieves significant improvement over the original LLaMA3 model, demonstrates comparable extraction capabilities to GPT-3.5’s structured mode, and achieves higher accuracy in retrieving API parameters.
- We export the converted extracted API information into Python functions as AI-usable tools that are compatible with LLM frameworks like LangChain [9]. To handle complex API responses, we developed an **evaluation pipeline** (Section 4.3) to verify the tools’ functionality and diagnose errors when detected (Section 5.2). We identified invalid parameter values as the primary cause of tool failures. To address this, we proposed a

<sup>1</sup>Code available at Link

method leveraging a knowledge base to **improve parameter value inference** (Section 4.4).

- To further demonstrate the effectiveness of our approach, we used the generated tools to develop a Tool Agent to support glycomaterials research as a case study (Figure 4). We collected relevant REST APIs contributed by researchers in the field and applied ToolFactory to them. To address the challenge of finding the proper parameter value such as database ID, we used information from verified tools to build a parameter database and refined other tools which have missing or imprecise information. As a result, we successfully **generated 92 verified tools** that enable access to and processing of glycan-related data, covering a wide range of tasks and glycan representations. Our findings also highlight the domain-agnostic nature of ToolFactory, as it can be effectively applied across diverse fields.

## 2 Related Works

**LLM for universal information extraction** Structured information extraction is important as it builds bridge between human-readable formats into machine-usable data. Lu et. al.[10] proposed a universal text-to-structure framework for universal information extraction(UIE) using transformers, proving the potential of language models in encoding complex structures and producing structured outputs. Dagdelen et. al. [11] finetuned LLAMA2 [12] and GPT-3 [13] to extract the desired information from scientific text that can be fit into defined JSON schema. Zhu et. al. [14] proposed a multi-agent approach, incorporating GPT4 and GPT3.5 for automatic knowledge graph extraction and reasoning. OpenAI released GPT4o and GPT4o-mini structured mode, which supports generating structured outputs in an update in July [15]. It forces GPT to follow the provided Pydantic structure or JSON schema. In this work, we employed both OpenAI model for data synthesis and finetuned LLAMA3 model for an opensource information extraction solution.

**Tool agents** LLM-based tool agents are able to reason through user queries, select and apply appropriate actions, and return the results of the chosen action. For example, Bran et. al. [5] develop ChemCrow by integrating GPT-4 and 18 tools designed by experts. It was demonstrated that ChemCrow was able to effectively automate a variety of chemical tasks. Qin et. al. [6] collected 16k public REST APIs from the RapidAPI platform [16], and trained a tool retriever that can choose the most appropriate API in response to a user query. However, their tool-building approach relied on RapidAPI, limiting adaptability to other platforms without significant human

effort. Similarly, RESTgpt[17] used OpenAPI Specification (OAS), restricting its applicability to APIs that provide OAS. Our goal is to create a universal tool-generation pipeline that works with any natural language API documentation.

## 3 API Extraction Benchmark Dataset

Previous research on tool agents, such as ToolLLM, has focused primarily on instructing LLM agents to use AI-usable tools and develop solution paths to answer user queries. Their AI-usable tools can be easily developed from APIs, supported by well-structured documentation, using relatively simple scripts. However, they did not focus on the challenges of developing AI-usable tools from APIs with less structured documentation. To fill this gap, we collected such APIs from APIList.com. We introduce our dataset as follows:

- **Documentation styles** The API documentation we collected can be categorized into three levels based on the organization and clarity of the API descriptions: (1) **Fully organized** The documentation follows a well-defined template, providing all necessary information to call the API in a structured and comprehensive way. Use cases are clearly explained, often with example code. API documentation on platforms like RapidAPI Hub and Postman API typically fall into this category. (2) **Semi-organized** This type of documentation includes basic descriptions but lacks clarity for each endpoint. Some essential information may not be labeled with specific keywords and is instead embedded within general text. Additional effort is often required to identify key details. (3) **Unorganized** These documents are minimal, often missing example code or detailed descriptions. They require some level of inference and reasoning to understand the API’s usage, with clues only available through endpoint names. We show that our benchmark consists of mostly semi-structure documentations, and only a few documentations are fully organized(Appendix B.1).
- **Selection criteria** We filtered for API documents that do not require API keys, allowing for easier and more convenient API access without needing authentication, which often involves submitting forms or linking payment methods. Although automating API key sign-up is feasible using web agents [18], we opted for APIs without authentication requirements to streamline the process. In total, 347 unique API documents were selected and downloaded in HTML format. Since some links pointed to index pages or API information that was dynamically loaded via JavaScript,

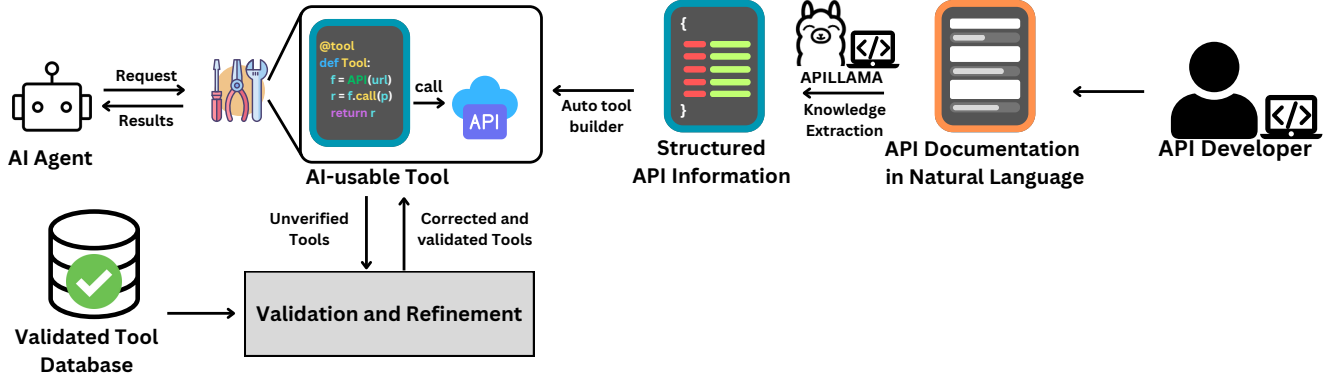


Figure 2: TooFactory: Automated pipeline for generating AI-usable tools from API documentations. The API documentations in free-text are processed by APILLama to extract structured information, which is used to build tools that interact with the corresponding APIs.

we employed a large language model to identify pages containing static HTML code with API endpoints. This approach ensured that we captured only the documentation with accessible and actionable API details.

Due to variations in the quality and completeness of API documentation, we extracted only the essential information needed for tool generation. Specifically, for each API documentation, we captured the base URL and a list of endpoints. For each endpoint, we extracted the endpoint path, required parameters, optional parameters, and a brief description. In cases where the base URL was not specified, human annotation was necessary. The detailed schema used for this extraction is provided in the Appendix A.

To implement this extraction, we defined the schema using a Pydantic model and employed GPT-4o in structured mode to parse the HTML documents and extract the desired information. After filtering out pages that lack API information (primarily product index pages), we obtained 167 API documentation with 744 endpoints and extracted their structured information in JSON format. An example of input API documentation and output JSON structure is in Appendix B.2.

## 4 ToolFactory Pipeline

### 4.1 APILLAMA

We fine-tuned the LLAMA3 model to develop an open-source solution for extracting structured information from API documentation. Recent LLM-based structured information extraction approaches rely on in-context learning, which involves providing a detailed JSON schema description or a one-shot input-output example. These contexts must be included in every

inference, leading to a significant overhead of redundant input tokens. Our approach is straightforward and efficient—since we focus on extracting information based on a fixed schema(Appendix A), we employ soft prompt tuning to encode the task, compressing the JSON schema into fewer tokens(Appendix F). This method significantly reduces token overhead during inference, as we compressed the original instruction which consisted of approximately 572 tokens (18 for the instruction and 554 for the JSON schema definition), into just 20 virtual tokens.

Despite its simplicity, our approach achieves competitive results. Even a fine-tuned 8B model demonstrates performance comparable to OpenAI’s GPT models(Section 5.1). Prompt tuning also addresses the challenge of data availability; after filtering, the number of public, free APIs is relatively small, making smaller fine-tuned models less prone to overfitting.

We used 20 virtual tokens as the trainable prompt and applied 4-bit quantization to the main model, optimizing memory usage and training efficiency. We used Adam optimizer with learning rate 0.001. The training experiment was conducted on an NVIDIA A40 GPU and completed in 2 hours.

### Task Formalization

APILLAMA aims to generate the structured annotation of an API from its documentation. Specifically, given the embedding of an API documentation  $X \in \mathbb{R}^{d \times n_X}$  that contains  $n_X$  tokens with hidden dimension  $d$ , a pre-defined output schema  $S$ , a trainable instruction prompt  $\varphi \in \mathbb{R}^{d \times n_I}$ , a LLM  $f_\theta$  with frozen weight  $\theta$ , the goal is to predict the corresponding structured information  $Y_{(n_Y)} = (y_1, y_2, \dots, y_{n_Y}) \in \mathbb{R}^{d \times n_Y}$ . Basically, we want to maximize the probability of conditional generation  $P_\varphi(\tilde{Y}|X; S; f_\theta)$ . For a LLM  $f_\theta$ , we used the

negative log likelihood loss for model training:

$$\varphi = \underset{\varphi}{\operatorname{argmin}} - \frac{1}{n_Y} \sum_{i=1}^{n_Y} \mathcal{L}(y_i)$$

$$\mathcal{L}(y_i) = \log P_{\varphi}(y_i | Y_{(i-1)}, X, S, f_{\theta})$$

where  $Y_{(i-1)}$  contains the ground truth tokens  $y_1, y_2, \dots, y_{i-1}$ . In testing, the ground truth will not be provided to the model. Instead, we use an auto-regressive method to predict the tokens:

$$\tilde{y}_i = \underset{y_i}{\operatorname{argmax}} P_{\varphi}(y_i | \tilde{Y}_{(i-1)}, X, S, f_{\theta})$$

$\tilde{Y}_{(i-1)}$  represents all previously predicted tokens. During implementation, the model generates a special `<EOS>` token to signal the end of generation, which is masked and excluded from attention calculations.

### Evaluation of Structured Information Extraction

While we trained the model as a text generation task, our ultimate goal is to extract the proper knowledge from any given API documentation. The token level prediction alone cannot reveal the performance of the model. Hence, we evaluated several aspects of the generated JSON files:

- **Structure Correctness:** We first validate if the generated JSON file matches with our predefined schema. The **Valid Ratio** metric evaluates the model’s overall ability to generate correctly structured JSON files.
- **Semantic Similarity:** The name and description for the API endpoint and API parameters are crucial for enabling the AI agent to understand how to use the tool correctly and effectively. To evaluate the model’s ability to extract precise semantic information, we embed the endpoint name, endpoint description and parameter description using *st-codesearch-distilroberta-base*[19], a sentence transformer trained on *CodeSearchNet*[20]. This embedding model is designed to handle both natural language and coding-related information. We then calculate the cosine similarity between the prediction and ground-truth embeddings for three metrics: **Endpoint Name Similarity**, **Endpoint Description Similarity**, and **Parameter Description Similarity**.
- **Functional Parameter Accuracy:** The API server validates the payload and processes parameters based on JSON keys. An invalid parameter name can fail an API call. Additionally, API methods (e.g., GET, POST) and parameter types (e.g., string, int) are critical components that can

be categorized. We therefore measure **Parameter Precision and Recall**, **Method Accuracy**, and **Parameter Type Accuracy**.

The experiment results (Section 5.1) show that APILLAMA performed nearly perfect (97%) in generating correct structures and is comparable with proprietary models in semantic similarity and value matching.

### 4.2 Tool-generation Process

Once the necessary API information is extracted, the extracted JSON files are processed by scripts to generate AI-usable tools. These tools are designed to adapt the API’s functionality into formats compatible with popular agent-building frameworks. We support exporting tool functions as Python functions, which can be directly integrated into open-source frameworks like LangChain[9] and OpenAgents[21]. Additionally, tools can be exported as OpenAPI YAML files, enabling seamless import into systems like OpenAI GPTs [22]. The details of the tool generation algorithm, including format adaptation and compatibility handling, are provided in Appendix C.

### 4.3 Tool Validation

An evaluation process is employed to validate each generated tool. Only tools that pass this evaluation will be made available for AI agents. We test a tool using example parameter values and consider it valid only if: (1) it returns a status code of 200, and (2) its content is neither empty nor an error message, as verified by an GPT-4o based evaluator. Otherwise, the tool is considered to have failed validation. We categorize the tool errors into 6 types: Missing Endpoint Path, Missing Base URL, Failed Validation, Abnormal Response, No Parameter Value and Wrong Parameter Value. A detailed explanation can be referred in Appendix E

In addition, we categorize error causes into four main categories: **C1** Missing API Documentation Details, **C2** Incorrectly Extracted URL Path, **C3** Incorrect Parameter Values, and **C4** Server-Side Errors. For each category, we provide a range of possible error diagnosis, from the most conservative to the most aggressive (see Appendix E1). We find that most of the errors are caused by **C3**, which inspires us to develop an better approach to produce high-quality parameter values.

### 4.4 Parameter Value Inference

API documentations vary in quality, and parameter value specifications are often missing, especially in poorly documented APIs. To tackle this problem, we propose the following method to infer missing parameter value specifications of an API from validated tools in the same application domain.

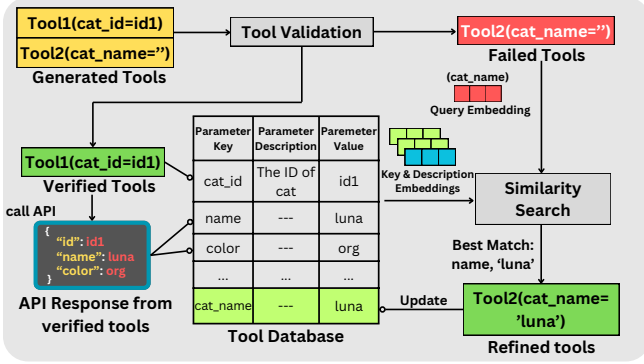


Figure 3: A parameter database is constructed using validated tools, enabling parameter value inference based on the semantic similarity of parameter keys and descriptions.

First, we built a knowledge base about parameters from validated tools. Each parameter entry contains an embedding vector indicating its semantics. Such knowledge can be extracted from the corresponding API documentations. Alternatively, we can extract such knowledge from the JSON responses of successfully executed tools. If a parameter, say  $\gamma$  is associated with a description (but not sufficient to determine its value), we embed its name and description separately. Using these embedding vectors, we retrieve the top 10 most similar parameters from the knowledge base—5 based on description cosine similarity and 5 based on parameter key similarity. Then we apply the value of each retrieved parameter to test the tool. If a candidate value passes the validation, we will use it to annotate  $\gamma$ , and save it to the parameter knowledge base. Figure 3 demonstrates our approach.

## 5 Results

Using the API Extraction Benchmark, we evaluated the capability of APILLAMA to extract API information from documentations in natural language and tested the automated tool generation process. We also demonstrated a case study where we built an AI-agent using the tools automatically generated by APILLAMA.

### 5.1 Structured API Knowledge Extraction via APILLAMA

We investigated the feasibility of combining LLM and soft prompt for extracting API knowledge.

**Model** APILLAMA consists of the LLAMA3-8B-instruct model, which is frozen, and 20 trainable soft prompt tokens. In addition, we extend the token limit from 8,192 to 10,240 to handle longer documents.

**Training setting** The input of the model is the text in an API documentation, and the output is the corresponding structured information in JSON. We ran-

domly split the API Extraction Benchmark into 80% for training and 20% for testing. The test set contains 34 API documents and 117 different endpoints. More setting details are provided in Appendix F.

**Metrics** See Section 4.1.

**Baselines** We chose the original LLaMA3-8B-instruct and GPT3.5 as the baseline models. We tested how the supporting information will affect the model performance. In the "one-shot learning" setting, we include an example in the input prompt that consists of an API document as the input along with its corresponding JSON annotation as the target. On the other hand, we defined the target JSON schema with Pydantic[23] package, which can be used by the function calling model[24] of GPT-3.5. This guarantees GPT-3.5 model to understand and produce the desired JSON structure.

The results are shown in Table 1. APILLAMA demonstrates strong performance, achieving competitive or superior scores across most metrics. In contrast, the vanilla LLaMA3 model performs poorly even when provided with a one-shot example. APILLAMA significantly improves the retrieval of API endpoints and the generation of accurate API annotations. Compared to GPT-3.5 in structured mode, APILLAMA retrieves fewer endpoints but excels in accurately extracting endpoint descriptions and parameters. The limited number of retrieved endpoints is likely due to the reasoning capability of the base LLaMA3-8B model, as its weights were not modified during fine-tuning. We attribute APILLAMA’s performance improvements to two key factors: (a) The soft prompt effectively describes the structured information extraction task, allowing the model to better understand the schema; (b) By minimizing the number of tokens used for instructions, APILLAMA enables its base model to focus more on the actual data within the prompt.

### 5.2 Automatic Tool Generation

We evaluated the performance of ToolFactory in generating AI-usable tools. These tools were derived from the structured API information extracted in Section 5.1 and tested following the procedure described in Section 4.3. Each tool was implemented as a Python function that takes an API endpoint and its corresponding parameters as inputs, processes the request, and returns the API response. An example of such a tool is provided in Appendix D.

The results are summarized in Table 2. While APILLAMA extracted fewer endpoints—likely due to the reasoning limitations of its base model—it produced the highest number of tools that passed validation. In the API Extraction Benchmark, 50 tools generated from ground-truth JSON files passed the validation test. APILLAMA was able to generate 26 tools (successful rate of 52%).

| Model                  | JSON Schema | API Endpoint |                 |                        | Method      | Parameter   |             |                        |               |
|------------------------|-------------|--------------|-----------------|------------------------|-------------|-------------|-------------|------------------------|---------------|
|                        | Valid Ratio | # Matched    | Name Similarity | Description Similarity | Accuracy    | Precision   | Recall      | Description Similarity | Type Accuracy |
| LLaMA3                 | 0           | 0            | 0               | 0                      | 0           | 0           | 0           | 0                      | 0             |
| LLaMA3+one shot        | 0.59        | 31           | <b>0.98</b>     | 0.88                   | 0.98        | 0.84        | 0.82        | 0.88                   | <b>1.00</b>   |
| GPT3.5+one shot        | 0.65        | 44           | 0.90            | 0.87                   | 0.98        | 0.88        | 0.87        | <b>0.93</b>            | <b>1.00</b>   |
| GPT3.5 structured mode | <b>1.00</b> | <b>84</b>    | 0.96            | 0.80                   | 0.97        | 0.83        | 0.82        | <b>0.93</b>            | 0.63          |
| <b>APILLAMA(Ours)</b>  | 0.97        | 66           | 0.94            | <b>0.89</b>            | <b>1.00</b> | <b>0.92</b> | <b>0.92</b> | 0.91                   | <b>1.00</b>   |

Table 1: A comparison of API information extraction result. Larger metric values indicate better performance. Metric explanation can be referred at Section 4.1

We also analyzed the errors (Appendix Table G2) in the generated tools. The primary cause of failures in APILLAMA and GPT-3.5 structured mode stemmed from incorrect parameter values. Notably, 30% to 55% of the tools generated by APILLAMA failed due to improper parameter values. This analysis highlights a key limitation of the current LLM-based approach and motivates us to incorporate domain knowledge to augment parameter values in the next experiment.

|                          | # Extracted tools | # Passed Tools |
|--------------------------|-------------------|----------------|
| LLaMA3+one shot          | 31                | 3              |
| GPT3.5+one shot          | 44                | 19             |
| GPT3.5 structured mode   | 84                | 25             |
| <b>APILLAMA(Ours)</b>    | 66                | <b>26</b>      |
| API Extraction Benchmark | 117               | 50             |

Table 2: Result of tool validation

### 5.3 Case Study: Automated Tool Agent Generation for Glyco Research

We applied ToolFactory to generate tools to support Glyco research and built an autonomous tool agent as the proof-of-principle. We collected the REST APIs of the most frequently used glyco databases, including **GlycoData** [25], **GlyGen** [26], **GlyTouCan** [27], **KEGG GLYCAN** [28], **Glycosmos** [29], **Glyconnect** [30], **The O-GlcNAc Database** [31], **GLYCAM** [32], **Protein API** [33], **PubChem** [34] and **UniLectin** [35]. In total, 92 automatically generated tools passed validation, their functionalities cover data searching, calculation, format translation, correction, and visualization (Figure 4).

#### Contribution to Glyco Research Community

Our AI agent provides natural language interfaces for researchers to access web services (e.g., databases and utility APIs) without requiring technical expertise, which greatly facilitates the usage of online resources and benefits researchers in several ways:

**(1) Automatic cross-database integration** No single database fulfills all information needs due to their specific focuses. For example, GlyTouCan catalogs glycan structures, KEGG GLYCAN maps path-

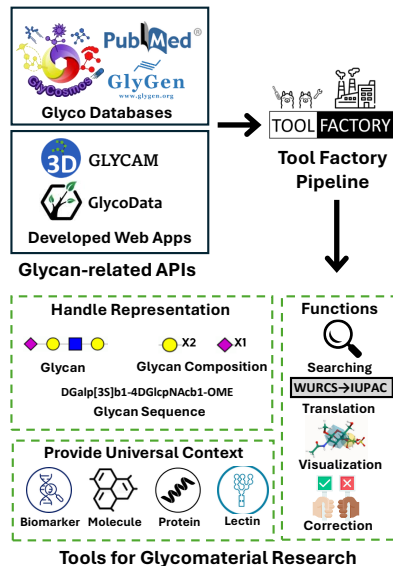


Figure 4: **AI Agent for Glycomaterial Research with Automated Tool Generation** By automating tool generation, the AI agent simplifies database access and supports glycan-related tasks such as searching, drawing, and format conversion. ToolFactory generated 92 validated AI-usable tools for various tasks across several databases. A web demo is developed using the OpenAgents framework.

ways and reactions, and PubChem offers general molecular data. Our AI agent integrates these diverse sources for seamless information access.

**(2) Tool synergy** Individual tools are often limited to specific scenarios, but our AI agent enhances their applicability by integrating them into cohesive workflows, including ID conversion, database querying, data normalization, visualization, and resolution of inconsistencies. Appendix Table H3 shows an example of a glycan being represented using various formats. This integration broadens the applicability of certain tools. For instance, the GLYCAM 3D visualization tool, initially limited to GLYCAM strings, now supports multiple glycan formats through automated conversion.

**(3) Facilitate the development and adoption of new services** Using ToolFactory, our AI agent can easily adopt new APIs and datasets, keeping its tools up-to-date. This enables researchers to prioritize scientific exploration over technical integration. In addition, the AI agent can serve as a platform for developing, sharing, and publishing applications, fostering dissemination and collaboration.

In the following, we highlight the technical details for solving the challenge of insufficient domain knowledge, which is common when developing domain-specific agent. For example, databases may use different reference IDs (see Appendix Table H4 for an example), requiring the agent to distinguish and pick the correct ID carefully. However, the information may not be covered in the API documentation, or is simply referred to as "ID" without any further information. The parameter value inference method proposed in Section 4.4 was deployed to address this problem, and its effectiveness is evaluated below.

**Setting** We used 92 validated tools in this experiment. We first built a parameter database utilizing the parameter examples of verified documents and response JSONs. For each parameter name-value pair, we added the information of its original API documentation to track the source. To simulate a scenario where the information about setting the value of a parameter is missing, we deliberately removed the parameter value information to test the inference method. We also restricted the access to any other information in the source document of that parameter. Each method will generate candidate settings for the parameters. An inference result is deemed successful if it allows the tool to pass the validation.

**Our parameter inference method** We retrieve five parameters which have the most similar description semantic similarity and another five parameters with highest parameter key similarity, resulting in at most 10 candidate parameters for each query parameter. We filter out candidates with similarity smaller than 0.5. If an API has multiple parameters, we first generate candidate values for each parameter, and sample the best 20 value combinations to test the tool.

**Parameter inference using GPT-4o** We prompt GPT-4o to predict candidate parameters. To ensure fair comparison, we allow GPT-4o to make up to 10 candidate guesses per tool, as our approach requests 10.06 times per tool on average. The failed guesses will be provided in the prompt, ensuring GPT-4o to generate diverse values.

**Results** Using the leave-one-API-out test setting, our approach successfully infers parameter values for 33 tools. Meanwhile, GPT-4o managed to find the parameter values for only 17 tools. The results clearly indicate the superiority of our approach over GPT-4o.

## 6 Conclusion

In this work, we designed a pipeline, **ToolFactory**, that can automatically convert natural language REST API documentation into tools usable by AI agents. This pipeline not only translates natural language into agent-compatible tools but also diagnoses and optimizes non-standard or inaccurate information in REST API documentation. We first built the API Extraction Benchmark, which collects API documentation of varying quality and standardizes it into JSON format. Next, we trained APILLAMA using prompt tuning, enabling it to extract structured information from the documentation effectively. Our experimental results demonstrate that APILLAMA achieves strong performance in information extraction, with the number of tools generated being comparable to GPT-based models. We identified inaccurate parameter values as a key factor affecting APILLAMA's performance and proposed a method to enhance parameter quality by constructing a domain-specific knowledge base. Finally, we applied ToolFactory to glycomaterial research and successfully developed an AI agent. Our work facilitates the creation of domain-specific agents, helping developers reduce the development and learning costs associated with these APIs.

## References

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, Mar. 2024.
- [2] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang, "Researchagent: Iterative research idea generation over scientific literature with large language models," 2024.
- [3] M. Abbasian, I. Azimi, A. M. Rahmani, and R. Jain, "Conversational health agents: A personalized llm-powered agent framework," 2024.
- [4] Y. Li, Y. Yu, H. Li, Z. Chen, and K. Khashanah, "Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance," 2023.
- [5] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, "Chemcrow: Augmenting large-language models with chemistry tools," 2023.
- [6] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "Toollm: Facilitating large language models to master 16000+ real-world apis," 2023.

- [7] Abhimanyu et al., "The llama 3 herd of models," 2024.
- [8] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," 2021.
- [9] C. Harrison, "Langchain." <https://github.com/langchain-ai/langchain>, 2024. Accessed: 2024-10-02.
- [10] Y. Lu, Q. Liu, D. Dai, X. Xiao, H. Lin, X. Han, L. Sun, and H. Wu, "Unified structure generation for universal information extraction," 2022.
- [11] J. Dagdelen, A. Dunn, S. Lee, N. Walker, A. S. Rosen, G. Ceder, K. A. Persson, and A. Jain, "Structured information extraction from scientific text with large language models," *Nature Communications*, vol. 15, no. 1, p. 1418, 2024.
- [12] e. a. Hugo Touvron, "Llama 2: Open foundation and fine-tuned chat models," 2023.
- [13] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020.
- [14] Y. Zhu, X. Wang, J. Chen, S. Qiao, Y. Ou, Y. Yao, S. Deng, H. Chen, and N. Zhang, "Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities," 2024.
- [15] OpenAI, "Openai structured outputs." <https://platform.openai.com/docs/guides/structured-outputs/introduction>, 2024. Accessed: 2024-10-02.
- [16] RapidAPI, "Rapidapi hub." <https://rapidapi.com/hub>, 2024. Accessed: 2024-12-31.
- [17] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, H. Huang, C. Li, K. Wang, R. Yao, Y. Tian, and S. Li, "Restgpt: Connecting large language models with real-world restful apis," 2023.
- [18] axiom.ai, "Automate logins with browser bots." <https://axiom.ai/automate/login>, 2024. Accessed: 2024-10-02.
- [19] N. Reimers, "st-codesearch-distilroberta-base," 2021.
- [20] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.
- [21] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu, T. J. Hua, J. Zhao, Q. Liu, C. Liu, L. Z. Liu, Y. Xu, H. Su, D. Shin, C. Xiong, and T. Yu, "Openagents: An open platform for language agents in the wild," 2023.
- [22] OpenAI, "Introducing gpts." <https://openai.com/index/introducing-gpts/>, 2024. Accessed: 2024-10-02.
- [23] S. Colvin, E. Jolibois, H. Ramezani, A. G. Badaracco, T. Dorsey, D. Montague, S. Matveenko, M. Trylesinski, S. Runkle, D. Hewitt, A. Hall, and V. Plot, "Pydantic," 01 2025. If you use this software, please cite it as described.
- [24] OpenAI, "Function calling." <https://openai.com/index/function-calling-and-other-api-updates/>, 2025. Accessed: 2025-01-10.
- [25] F. Wang, S. Seth, S. R. Ramacharla, and S. A. Deshmukh, "Glycodata." [glycodata.org/](https://glycodata.org/), 2025. Accessed: 2025-1-15.
- [26] W. S. York, R. Mazumder, R. Ranzinger, N. Edwards, R. Kahsay, K. F. Aoki-Kinoshita, M. P. Campbell, R. D. Cummings, T. Feizi, M. Martin, et al., "Glygen: computational and informatics resources for glycoscience," *Glycobiology*, vol. 30, no. 2, pp. 72–73, 2020.
- [27] M. Tiemeyer, K. Aoki, J. Paulson, R. D. Cummings, W. S. York, N. G. Karlsson, F. Lisacek, N. H. Packer, M. P. Campbell, N. P. Aoki, et al., "Glytoucan: an accessible glycan structure repository," *Glycobiology*, vol. 27, no. 10, pp. 915–919, 2017.
- [28] K. Hashimoto, S. Goto, S. Kawano, K. F. Aoki-Kinoshita, N. Ueda, M. Hamajima, T. Kawasaki, and M. Kanehisa, "Kegg as a glycome informatics resource," *Glycobiology*, vol. 16, no. 5, pp. 63R–70R, 2006.
- [29] I. Yamada, M. Shiota, D. Shinmachi, T. Ono, S. Tsuchiya, M. Hosoda, A. Fujita, N. P. Aoki, Y. Watanabe, N. Fujita, et al., "The glycosmos portal: a unified and comprehensive web resource for the glycosciences," *Nature Methods*, vol. 17, no. 7, pp. 649–650, 2020.
- [30] D. Alocci, J. Mariethoz, A. Gastaldello, E. Gasteiger, N. G. Karlsson, D. Kolarich, N. H. Packer, and F. Lisacek, "Glyconnect: glycoproteomics goes visual, interactive, and analytical," *Journal of proteome research*, vol. 18, no. 2, pp. 664–677, 2018.
- [31] E. Wulff-Fuentes, R. R. Berendt, L. Massman, L. Danner, F. Malard, J. Vora, R. Kahsay, and S. Olivier-Van Stichelen, "The human

o-glcnacone database and meta-analysis," *Scientific data*, vol. 8, no. 1, p. 25, 2021.

- [32] Woods Group, "Glycam web: Website builders," 2025. Accessed 2025.
- [33] A. Nightingale, R. Antunes, E. Alpi, B. Bursteinas, L. Gonzales, W. Liu, J. Luo, G. Qi, E. Turner, and M. Martin, "The Proteins API: accessing key integrated protein and genome information," *Nucleic Acids Research*, vol. 45, pp. W539–W544, 04 2017.
- [34] S. Kim, P. A. Thiessen, E. E. Bolton, J. Chen, G. Fu, A. Gindulyte, L. Han, J. He, S. He, B. A. Shoemaker, *et al.*, "Pubchem substance and compound databases," *Nucleic acids research*, vol. 44, no. D1, pp. D1202–D1213, 2016.
- [35] A. Imberty, F. Bonnardel, and F. Lisacek, "Unilectin, a one-stop-shop to explore and study carbohydrate-binding proteins," *Current Protocols*, vol. 1, no. 11, p. e305, 2021.

## A API-extraction Schema

We defined the JSON schema to extract API information. In practice, this is achieved by defining the pydantic object of API objects.

```
1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "definitions": {
4     "Parameters": {
5       "type": "object",
6       "properties": {
7         "name": {
8           "type": "string",
9           "description": "Name of the parameter"
10        },
11        "type": {
12          "type": "string",
13          "description": "Type of the parameter"
14        },
15        "description": {
16          "type": "string",
17          "description": "Description of the parameter. If the parameter is categorical,
18            please list all possible values."
19        },
20        "default": {
21          "description": "Default value of the parameter"
22        },
23        "example": {
24          "description": "Example value of the parameter"
25        }
26      },
27      "required": ["name"]
28    },
29    "Endpoint": {
30      "type": "object",
31      "properties": {
32        "name": {
33          "type": "string",
34          "description": "Name of the endpoint"
35        },
36        "description": {
37          "type": "string",
38          "description": "Description of the endpoint"
39        },
40        "method": {
41          "type": "string",
42          "description": "Method of the endpoint"
43        },
44        "url": {
45          "oneOf": [
46            { "type": "string" },
47            { "type": "array", "items": { "type": "string" } }
48          ],
49          "description": "URL of the endpoint, start with http:// or https://"
50        },
51        "headers": {
52          "type": "array",
```

```
52     "items": { "type": "string" },
53     "description": "Headers of the endpoint",
54     "default": []
55 },
56 "required_parameters": {
57     "type": "array",
58     "items": { "$ref": "#/definitions/Parameters" }
59 },
60 "optional_parameters": {
61     "type": "array",
62     "items": { "$ref": "#/definitions/Parameters" }
63 }
64 },
65 "required": ["name", "method", "url"]
66 },
67 "API": {
68     "type": "object",
69     "properties": {
70         "title": {
71             "type": "string",
72             "description": "Title of the API"
73         },
74         "endpoints": {
75             "type": "array",
76             "items": { "$ref": "#/definitions/Endpoint" }
77         }
78     },
79     "required": ["endpoints"]
80 }
81 },
82 "type": "object",
83 "properties": {
84     "API": { "$ref": "#/definitions/API" }
85 }
86 }
```

## B API Extraction Benchmark

### B.1 API Documentation Classification

We classify API documentations into 3 categories: **Fully organized**, **Semi-organized** and **Unorganized**, based on their clarity and completeness of information. Using GPT-4o with chain-of-thought reasoning (Appendix I.3), we categorized all API documentation in the API Extraction Benchmark. Out of the total, 24 documents were classified as Fully Organized, 134 as Semi-Organized, and 9 as Unorganized. Figure B1 provides examples of API documentation in each category. Notably, none of the documentation is fully standardized, and Semi-Organized and Unorganized documents frequently lack clarity or essential API details.

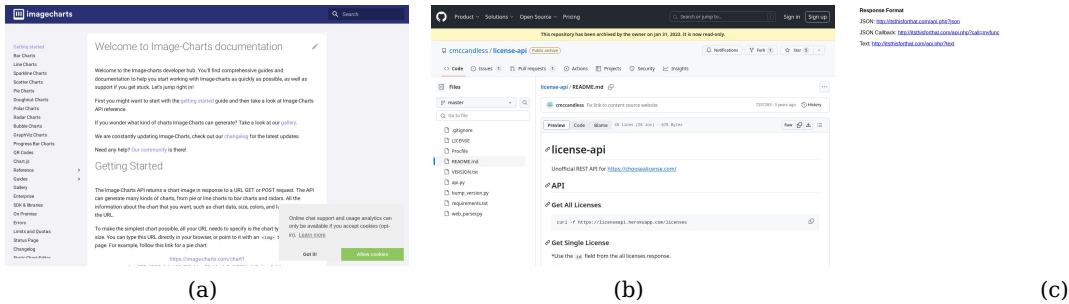


Figure B1: Example of API documentation of each category. (left) is an example of organized API documentation [link](#), (mid) is the example of semi-organized documentation [link](#), and (right) is the unorganized API documentation [link](#)

### B.2 Extraction Task Example

In API Extraction Benchmark, the input is the text from the API documentation(HTML) page, while the output is a JSON string following our API-extraction Schema. Below is the input example:

PokÃ©mon TCG Developers PokÃ©mon TCG Developers JOIN THE COMMUNITY OF DEVELOPERS BUILDING WITH THE POKÃ©

x89MON TCG API Next The most comprehensive dataset you can find From Base Set through Sword & Shield , the PokÃ©mon TCG API has the card data you need to build the ultimate application, whether that be mobile or web. Get access to prices, hi-res images, and more. Focus on your application and let us worry about the card data. Some features of the PokÃ©mon TCG API include: Prices by TCGplayer Hi-Res Images Detailed Search Queries Developer SDKs With a large community contributing to the project through various SDKs, development for you and your team has never been easier. In-depth details of every card All data delivered in JSON format such as types, attacks, abilities, and more. For Developers The PokÃ©mon TCG API allows developers to ingest card data programatically. The API exposes card and set details in an easy to consume format. Stay up to date with new sets, and access new data right away on launch day. Advanced Search Queries Perform advanced search queries with a familiar Lucene-like syntax. Find cards by name, type, release date, legality, and more. Use multiple types of operators to filter to specific cards. This API supports operators such as logical AND, OR, NOT, and range operators. View the documentation for more details. GET https://api.pokemontcg.io/v2/cards?q=name:gardevoir (subtypes:mega OR subtypes:vmax) Gain access to the PokÃ©mon TCG Developer Portal to start using the PokÃ©mon TCG API The PokÃ©mon TCG Developer Portal lets you manage your account and API Key associated with the PokÃ©mon TCG API. Creating an account will get you access to higher rate limits and no IP restrictions. Get Started Questions? We are here to help you. If you have general usage questions, it is highly recommended to ask your questions at community Discord channel. There are tons of devs who are more than willing to help you out with any questions regarding the API. If you are an individual or business who is interested in receiving higher rate limits for the API, please contact Andrew via email: [email xa0protected] . I try to respond to all inquiries within 72 hours. API Documentation PokÃ©mon TCG API Documentation PokÃ©mon Card Search Engine PokÃ©mon TCG Guru Donate Patreon , Kofi Discord https://discord.gg/dpsTCvg Email [email xa0protected] Elsewhere GitHub Twitter LinkedIn This website is not produced, endorsed, supported, or affiliated with Nintendo or The PokÃ©mon Company.

And the output example:

```
1 {
2   {
3     "title": "Pok\mon TCG API Documentation",
4     "endpoints": [
5       {
6         "name": "Search Cards",
7         "description": "Perform advanced search queries to find cards by name, type,
8           release date, legality, and more.",
9         "method": "GET",
10        "url": "https://api.pokemontcg.io/v2/cards",
11        "headers": [],
12        "required_parameters": [
13          {
14            "name": "q",
15            "type": "string",
16            "description": "The search query using Lucene-like syntax.",
17            "default": null,
18            "example": "name:gardevoir"
19          }
20        ],
21        "optional_parameters": []
22      }
23    ]
24  }
```

## C JSON-To-Tool generation

JSON-To-Tool generation is mostly about engineering. As the documentations are written in various formats, multiple conditions need to be considered, especially when handling URLs. It is common for the API server to only support a specific input pattern, which is not mentioned in the documentation or the error message. To make the autogenerated tool more robust, we considered the following procedures in our tool generator:

- **parameter handling** Documentations use various ways to represent path parameters. We set matching rules to find commonly used patterns such as ":param", "{param}", "<param>" etc.
- **encoding correction** To pass some special characters such as "+" or "=", the URL will use an encoding method known as percent encoding. Usually this won't be an issue, but for some APIs this need to be done before passing the parameters.
- **required parameter checking** As APIs may not necessarily return the error information, we added a required parameter validation step in the generated tool so that we can report any missing parameter errors to the AI agent.

## D Tool Example

By applying the JSON-To-Tool generation script, we can convert the structured API information into an executable tool. In this work, a tool is a python function which can be called by the LLM. A typical tool example for 'Pokemon TCG API' is shown below.

```
1 def search_cards(q=None):
2     api_url = f"https://api.pokemontcg.io/v2/cards"
3     querystring = {'q': q, }
4     assert q is not None, 'Missing required parameter: q'
5
6     response = requests.get(url=api_url, params=querystring, timeout=50, verify=False)
7     if response.status_code != 200:
8         response2 = requests.get(url=api_url, timeout=50) # in case API can't handle redundant params
9         response = response2
10    return response
11    # print(response.json())
```

```

12
13 if __name__ == '__main__':
14     r = search_cards(q='''name:gardevoir''')
15     r_json = None
16     try:
17         r_json = r.json()
18     except:
19         pass
20     import json
21     result_dict = dict()
22     result_dict['status_code'] = r.status_code
23     result_dict['text'] = r.text
24     result_dict['json'] = r_json
25     result_dict['content'] = r.content.decode("utf-8")

```

## E Tool Error Types and Causes

Errors are categorized below:

- **Incomplete URL:** Occasionally, the URL of a tool is incomplete, resulting in failed requests. These errors can be classified into two subtypes: **Missing Endpoint Path** or **Missing Base URL**. By examining the corresponding API documentations, we found that endpoint paths were usually provided, but base URLs were often missing.
- **Request Errors:** Request errors are complex and challenging to diagnose, as status codes alone do not clearly indicate whether the issue originates from the server or client side. A status code of 200 ("OK") guarantees valid communication between the client and server. To further validate the response content in such cases, we use an GPT-4o based evaluator (Appendix I.1). If the evaluator returns "pass", the tool is labeled as **Passed Validation**; otherwise, it is labeled as **Failed Validation**. For other status codes or unexpected cases, the tool is classified under **Abnormal Response**.
- **Incorrect Parameter Values:** If a tool throws an exception that does not fall into any of the previously mentioned scenarios, it indicates that the tool was very likely called with invalid parameters. Specifically:
  - If a required parameter is missing, the error is classified as **No Parameter Value**.
  - If all required parameters are provided but the tool still fails, the issue likely stems from an incorrect example value, and the error is classified as **Wrong Parameter Value**.

For all error types other than **Passed Validation**, we group them into four main categories: **C1** Missing API Documentation Details, **C2** Incorrectly Extracted URL Path, **C3** Incorrect Parameter Values, and **C4** Server-Side Errors. For each category, we provide a range of possible error diagnosis, from the most conservative to the most aggressive (see Appendix E).

| Category                          | Conservative Estimate                   | Aggressive Estimate(Additional Terms Only) |
|-----------------------------------|-----------------------------------------|--------------------------------------------|
| Missing API Documentation Details | 0                                       | Missing Base URL+No Parameter Value        |
| Incorrectly Extracted URL Path    | Missing Endpoint Path                   | Missing Base URL                           |
| Incorrect Parameter Values        | Wrong Parameter Value+Failed Validation | No Parameter Value+Abnormal Response       |
| Server-Side Error                 | 0                                       | Failed Validation+Abnormal Response        |

Table E1: Estimation of error causes

## F APILLAMA training

APILLAMA is fine-tuned on the basis of meta-llama/Meta-Llama-3-8B-Instruct. The default configuration has hidden dimension 4,096 and context length 8,192. We extend the context length to 10,240 to support longer API documentation input. A trainable soft prompt with 20 virtual tokens is added, with the weight initialized by the instruction prompt "You will be given an API documentation. Extract the API endpoints and output in JSON format.". To wrap up, APILLAMA has a total of  $20 \times 4096 = 81920$  trainable parameters. We use an AdamW optimizer with learning rate 0.001, a linear scheduler with warm-up steps 10 and negative log likelihood loss for training. We train the model with one epoch, 149 data points. Figure F2 shows the training loss curve for APILLAMA.

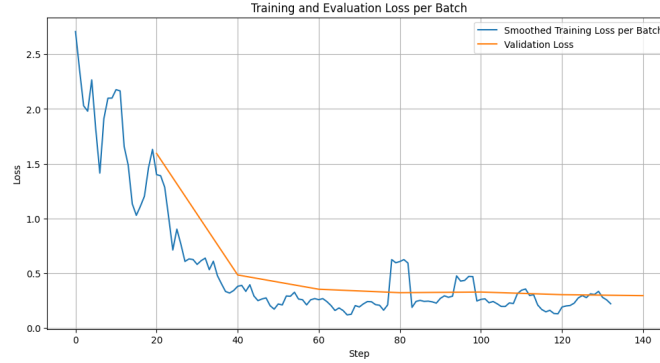


Figure F2: Training loss of APILLAMA. We applied moving average of window size 5 to smooth the training curve. Validation result is produced in every 20 steps.

## G Tool Validation Errors

Table G2 displays the errors occurred in Section 5.2.

|                        | Error Type            |                  |                   |                   |                    |                       |
|------------------------|-----------------------|------------------|-------------------|-------------------|--------------------|-----------------------|
|                        | Missing Endpoint Path | Missing Base URL | Failed Validation | Abnormal Response | No Parameter Value | Wrong Parameter Value |
| Ground Truth           | 0                     | 4                | 9                 | 23                | 14                 | 10                    |
| LLaMA3+one shot        | 0                     | 4                | 0                 | 2                 | 0                  | 4                     |
| GPT3.5+one shot        | 0                     | 1                | 4                 | 5                 | 0                  | 1                     |
| GPT3.5 structured mode | 0                     | 9                | 4                 | 21                | 14                 | 10                    |
| <b>APILLAMA(Ours)</b>  | 0                     | 0                | 4                 | 20                | 0                  | 16                    |

|                        | Error Cause             |                                |                            |                    |
|------------------------|-------------------------|--------------------------------|----------------------------|--------------------|
|                        | Missing API DOC Details | Incorrectly Extracted URL Path | Incorrect Parameter Values | Server-side Errors |
| Ground Truth           | 0-18                    | 0-4                            | 19-56                      | 0-32               |
| LLaMA3+one shot        | 0-4                     | 0-4                            | 4-6                        | 0-2                |
| GPT3.5+one shot        | 0-1                     | 0-1                            | 5-10                       | 0-9                |
| GPT3.5 structured mode | 0-23                    | 0-9                            | 14-47                      | 0-25               |
| <b>APILLAMA(Ours)</b>  | 0-0                     | 0-0                            | 20-36                      | 0-24               |

Table G2: Analysis of errors in generated tools. The top table shows the count of tools where certain each error type occurs. The bottom table shows an estimation of the cause for the errors, from the most conservative estimate to the most aggressive estimate.

## H Parameters in Glycoscience APIs

| String Representation |                                           |             |
|-----------------------|-------------------------------------------|-------------|
| IUPAC Condensed       | Fuc(a1-2)Gal(b1-3)[Fuc(a1-4)]GlcNAc(b1-   |             |
| GLYCAM                | LFucpa1-2DGalpb1-3[LFucpa1-4]DGlcNacb1-OH |             |
| Database ID           |                                           | Glycan Name |
| GlyToucan ID          | G00048MO                                  | Lewis b     |
| PubChem ID            | 45480569                                  |             |

Table H3: Example of different representations of Glycan *Lewis b*. The table shows various ways to represent the glycan, demonstrating the complexity of glycan reference.

| Database    | Reference ID | Primary Usage                                    |
|-------------|--------------|--------------------------------------------------|
| GlyTouCan   | GlyTouCan ID | Glycan Structure<br>GlyTouCan ID for API Calling |
| KEGG GLYCAN | KEGG ID      | KEGG Pathway<br>Reaction                         |
| GlyGen      | GlyTouCan ID | Publication<br>Cross Reference                   |
| O-GlcNAc    | UniProtKB ID | Protein O-GlcNAcylation Data                     |
| PubChem     | PubChem CID  | Chemical Properties                              |
| Unilectin   | Unilectin ID | Get Lectin and Ligand Information                |

Table H4: Glycan databases covered in glyco agent

## I LLM Prompts

### I.1 API Response Validation Prompt

```
1 '''
2 Decide if the following API response is an information or an error message.
3
4 API Description:
5 {description}
6
7 API Response:
8 {response}
9 '''
```

### I.2 Parameter Generation Prompt

```
1 '''
2 You will be provided with the information of an API and its parameters. The example values of the parameters are missing. You need to
  guess the parameter values.
3 You may have failed several times before. If you guess with similar values, you may fail again. Please be innovative and try different
  values and formats.
4
5 Your previous failed guesses:
6 ***history start
7 {history}
8 ***history end
9 '''
```

```

10 API Description:
11 {description}
12
13 Parameter Description:
14 {param_description}
15
16 Your Guess:
17 '''
18
19 # Output Class Structure
20 class Parameter(BaseModel):
21     parameter_key: str
22     parameter_guess: str = Field(..., description="The guessed values of the parameter.")
23
24 class ParameterList(BaseModel):
25     parameters: List[Parameter] = Field(..., min_items=1, description="The list of parameters and their guesses.")

```

### I.3 API Documentation Classification Prompt

```

1 '''
2 You need to group the API documentation with the following standards:
3
4 Fully Organized: The documentation follows a well defined template, most likely to be from an API platform. It is well-structured,
5 clear, and easy to understand. It includes detailed descriptions, example code, and explanations of how to use the API.
6 Semi-Organized: Lacks some structure, but still includes most of the necessary information. It may be missing some examples or
7 descriptions, making it slightly more difficult to understand how to use the API.
8 Unorganized: Missing example or description, or the structure is unclear, making it difficult to understand how to use the API.
9
10 ===
11 API Documentation:
12 {API_DOC}
13 '''
14
15 # Output Class Structure
16 class Classification(BaseModel):
17     analysis: str = Field(..., description="The analysis of the API documentation. Make it within 300 characters.")
18     category: str = Field(..., enum=["Fully Organized", "Semi-Organized", "Unorganized"])

```